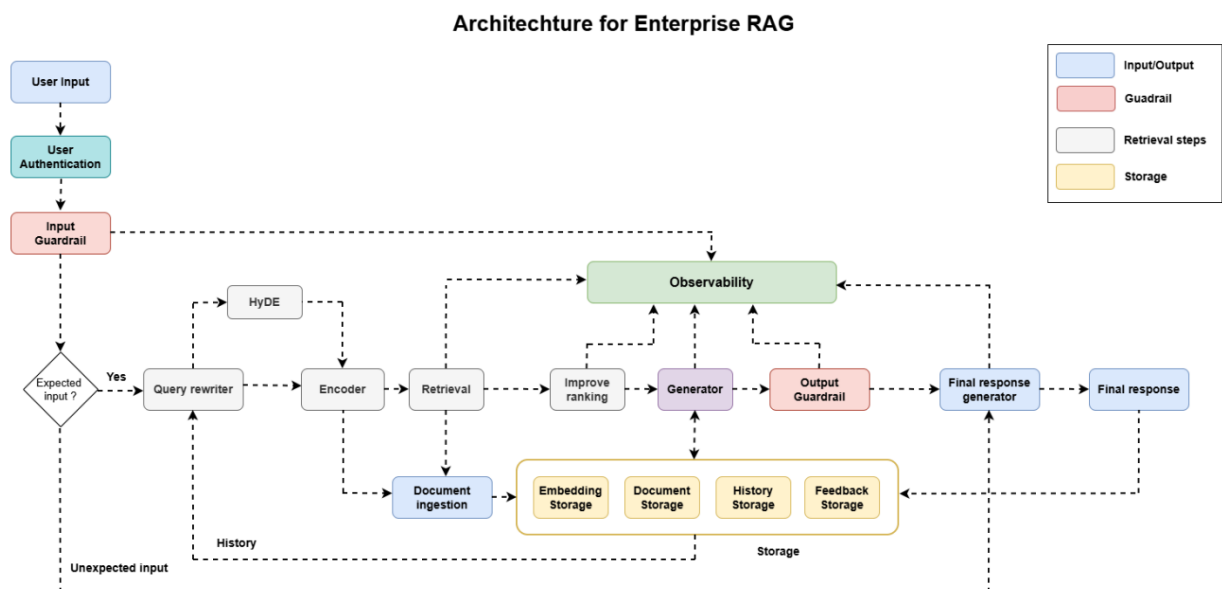


Build an end-to-end RAG Production Architecture with Docker, eval, and guardrails

So, here's a complete, end-to-end RAG architecture with:

- ✓ Dockerized microservices
- ✓ Evaluation pipeline
- ✓ Guardrails (input + output)
- ✓ Scalable retrieval + LLM layer

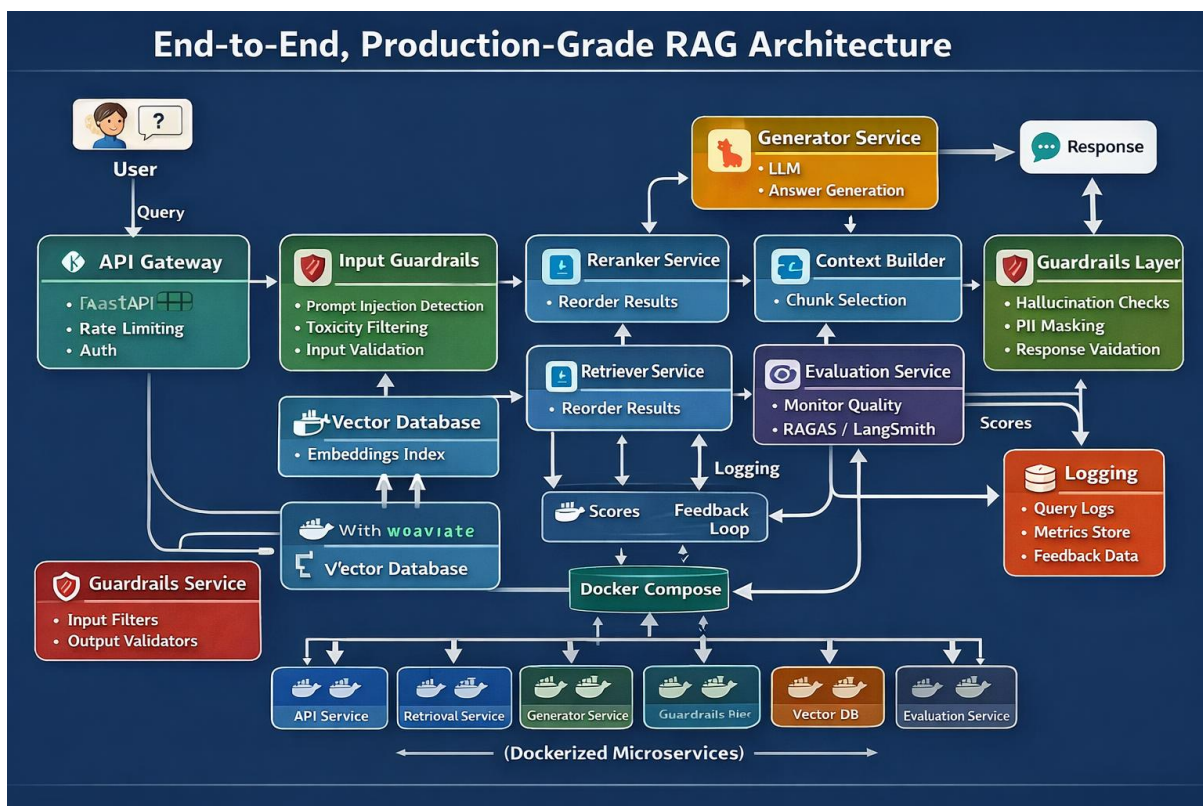


The diagram correctly includes:

- User flow
 - User input → Authentication → Input guardrail ✓
- Query processing
 - Query rewriting ✓
 - HyDE (Hypothetical Document Embeddings) ✓
 - Encoder → Retrieval ✓
- Retrieval pipeline
 - Ranking improvement ✓
 - Generator (LLM) ✓
- Guardrails

- Input guardrail ✓
- Output guardrail ✓
- **Storage layers**
 - Embeddings ✓
 - Documents ✓
 - History ✓
 - Feedback ✓
- **Observability layer**
 - Central monitoring ✓ (this is very important and often missed)

👉 So conceptually, this is a solid enterprise-grade RAG pipeline.



🧠 1. High-Level Architecture (Production View)

🔄 Flow

User → API Gateway → Guardrails (Input)
 → Retriever → Reranker → Context Builder
 → LLM (Generator)
 → Guardrails (Output)
 → Response

Parallel:

→ Logging → Evaluation → Feedback loop

2. Core Components

API Layer (FastAPI)

- Handles requests
- Routes to pipeline
- Auth + rate limiting

Retrieval Layer

- **Vector DB:** embeddings search
- **Hybrid search** (BM25 + vector)

👉 Options:

- FAISS (local)
- Weaviate
- Pinecone

LLM Layer

- Answer generation
- Can be local or API-based

👉 Example:

- Ollama (phi3, llama3)
- OpenAI / Claude (if cloud allowed)

Orchestration Layer

- Controls pipeline

👉 Tools:

- LangChain
- LlamaIndex

Guardrails Layer (CRITICAL)

Input Guardrails

- Prompt injection detection
- Toxicity filtering
- Schema validation

Output Guardrails

- Hallucination detection
- PII masking
- Response validation

Tools:

- Guardrails AI
- Rebuff

Evaluation Layer

Tracks:

- Answer correctness
- Faithfulness to context
- Latency
- Cost

Tools:

- RAGAS
- LangSmith

Storage Layer

- Document store (S3 / DB)

- Metadata store
- Logs



3. Dockerized Microservices Architecture



Services

```
rag-system/  
|  
├── api-gateway/      (FastAPI)  
├── retriever-service/ (embedding + search)  
├── llm-service/       (Ollama wrapper)  
├── guardrails-service/ (validation)  
├── evaluation-service/ (RAGAS)  
├── ingestion-service/ (document pipeline)  
├── vector-db/         (FAISS / Weaviate)  
└── docker-compose.yml
```



docker-compose.yml (Simplified)

```
version: '3.9'  
  
services:  
  
  api:  
    build: ./api-gateway  
    ports:  
      - "8000:8000"  
    depends_on:  
      - retriever  
      - llm  
      - guardrails  
  
  retriever:  
    build: ./retriever-service
```

llm:

image: ollama/ollama

ports:

- "11434:11434"

guardrails:

build: ./guardrails-service

evaluation:

build: ./evaluation-service

vector-db:

image: semitechnologies/weaviate

ports:

- "8080:8080"



4. End-to-End Request Flow (Detailed)

Step-by-step:

1. User Query

→ /ask

2. Input Guardrails

- Block:
 - prompt injection
 - malicious queries

3. Retriever

- Embed query
- Hybrid search
- Top-K docs

4. Reranker (optional but recommended)

- Improve relevance

5. Context Builder

- Chunk selection
- Token optimization

6. LLM Call

- Generate answer

7. Output Guardrails

- Check hallucination
- Enforce format

8. Response to User

9. Async Evaluation

- Store:
 - query
 - retrieved docs
 - answer

→ run RAGAS metrics



5. Evaluation Pipeline (Important for Production)

Metrics

- **Faithfulness**
- **Answer relevance**
- **Context precision**
- **Latency**

Example (Python)

```
from ragas import evaluate

result = evaluate(
    dataset,
    metrics=["faithfulness", "answer_relevancy"]
)
```

6. Guardrails Example

Input validation

```
def validate_input(query):
    if "ignore previous instructions" in query:
        raise ValueError("Prompt injection detected")
```

Output validation

```
def validate_output(answer):
    if "I don't know" not in answer and len(answer) < 5:
        return "Low confidence response blocked"
```

7. Production Enhancements

Performance

- Redis caching
- Async pipelines
- Streaming responses

Scalability

- Kubernetes deployment

- Horizontal scaling

Security

- Auth (JWT)
- Rate limiting
- Audit logs

8. Final Architecture (Mental Model)

RAG = Retrieval + Reasoning + Control + Evaluation

Layer	Responsibility
Retrieval	Find knowledge
Reasoning	Generate answer
Control	Prevent errors
Evaluation	Improve system

Real Industry Setup (What you should actually build)

Start with:

```
FastAPI
+ LangChain
+ FAISS
+ Ollama
+ Guardrails
+ RAGAS
+ Docker
```